PLAYTO PAY

Founding Engineer Assignment

Payout Engine — Complete Task Breakdown

Kya banana hai • Kaise banana hai • Test cases • Dhyan rakhne wali baatein

⏱ **Deadline: 5 Days**
⚡ **Expected Effort: 10–15 Hours**
💰 **Role: Founding Engineer | 6–10 LPA + ESOPs**

01 Task Ka Context — Kya Banana Hai

Playto Pay ek platform hai jo Indian agencies aur freelancers ko international payments collect karne mein help karta hai. Flow simple hai:

🌍 **International Customer (USD) → Playto Collect → Merchant (INR)**

Tumhara kaam hai ek Payout Engine banana — woh middle layer jahan merchants apna balance dekhte hain, payout request karte hain, aur payout ka status track karte hain.

Tech Stack (Fixed — Change Mat Karna)

Layer	Technology	Notes
Backend	Django 4.x + DRF	REST APIs
Database	PostgreSQL	Required — SQLite allowed nahi
Background Jobs	Celery + Redis	Sync code allowed nahi
Frontend	React + Tailwind	Vite recommended
Deployment	Railway / Render / Fly.io	Free tier chalega

02 Core Features — Kya Kya Banana Hai

Feature 1 — Merchant Ledger

Har merchant ka balance paise mein track hoga (integer, kabhi float nahi).

- → Balance store nahi hoga — hamesha transactions se calculate hoga
- → Do types of transactions: Credit (customer ne pay kiya) aur Debit (payout nikala)
- → 2-3 merchants seed karne hain credit history ke saath
- → Customer payment flow banana nahi — sirf seeding karni hai

> *BigIntegerField use karo — FloatField ya DecimalField bilkul mat use karna. Yeh assignment ki hardest rule hai.*

Feature 2 — Payout Request API

POST /api/v1/payouts/ — merchants yahan payout request karte hain.

- → Header mein Idempotency-Key (UUID) mandatory hai
- → Body mein: amount_paise (integer) aur bank_account_id
- → Payout "pending" state mein create hota hai
- → Funds hold ho jaate hain (available balance se minus)
- → Same key se dobara call karo → same response milega, duplicate nahi banta

Feature 3 — Background Payout Processor (Celery Worker)

Pending payouts uthata hai aur bank settlement simulate karta hai:

Outcome	Probability	Kya Hota Hai
Success ✅	70%	Payout completed. Funds final.
Failure ❌	20%	Payout failed. Held funds wapas merchant balance mein.
Hang ⏳	10%	Processing mein ruk jaata hai. Retry mechanism handle karta hai.

Feature 4 — React Merchant Dashboard

- → Available balance, held balance, recent transactions dikhao
- → Payout request karne ka form
- → Payout history table with live status (polling every 3 seconds)

03 Technical Constraints — Yahi Graded Hoga

Features banana easy hai. Yeh 5 constraints hain jo CTO actually check karega:

🔒 Constraint 1 — Money Integrity

Rule	Detail
Storage	BigIntegerField — sirf paise mein (100 rupees = 10000 paise)
Calculation	Database-level SUM() — Python mein rows fetch karke add mat karna
Invariant	SUM(credits) - SUM(debits) = displayed balance — hamesha match karna chahiye
Forbidden	FloatField, DecimalField, Python arithmetic on fetched rows

> *Balance ek derived value hai, stored column nahi. Yeh sochna ki balance ek "account mein rakha hua number" hai — galat hai. Sahi: balance = ledger transactions ka sum.*

⚡ Constraint 2 — Concurrency (Sabse Common Bug)

Scenario: Merchant ke paas 100 rupees hain. Do simultaneous 60 rupee requests aati hain. Exactly ek succeed karni chahiye, doosri reject honi chahiye.

> *Wrong approach: Python mein balance check karo, phir create karo — dono requests ek saath balance read karengi aur dono succeed karengi. Yeh race condition hai.*

> *Correct approach: SELECT FOR UPDATE NOWAIT (PostgreSQL row-level lock) — ek request lock leti hai, doosri immediately fail hoti hai bina wait kiye.*

Code pattern:

```
merchant =
Merchant.objects.select_for_update(nowait=True).get(id=merchant_id)
```

🔑 Constraint 3 — Idempotency

- → Idempotency-Key header merchant-supplied UUID hai
- → Same key se dobara call → exact same response, no duplicate payout
- → Keys merchant-scoped hain (alag merchant same key use kar sakta hai)
- → Keys 24 hours mein expire ho jaati hain
- → DB mein unique_together = [(merchant, idempotency_key)] constraint lagao

> *In-flight request edge case: Agar pehli request process ho rahi hai aur doosri same key se aati hai toh DB unique constraint second INSERT fail kar dega — catch karo aur 200 return karo.*

🔄 Constraint 4 — State Machine

Payout ke 4 states hain. Sirf ye transitions legal hain:

From State	To State	Legal?	Action
pending	processing	✅ YES	Worker ne pick up kiya
processing	completed	✅ YES	Bank success
processing	failed	✅ YES	Bank failed / timeout
completed	pending	❌ NO	Block karo — ValueError throw karo
failed	completed	❌ NO	Block karo — ValueError throw karo
completed	failed	❌ NO	Block karo — ValueError throw karo
Any backward	Any	❌ NO	Block karo

Implementation: ALLOWED_TRANSITIONS dictionary use karo. transition_to() method mein check karo — seedha .status = assign mat karna kabhi.

♻️ Constraint 5 — Retry Logic

- → Payouts jo 30 seconds se zyada "processing" mein hain → retry karo
- → Exponential backoff use karo
- → Maximum 3 attempts
- → 3 attempts ke baad → failed mark karo aur funds wapas karo atomically
- → Celery Beat se har 60 seconds mein stuck payouts check karo

04 Test Cases — Kya Kya Test Karna Hai

Minimum 2 tests required hain — concurrency aur idempotency. Yeh dono CTO khud check karta hai.

Test 1 — Concurrency Test (Most Critical)

Yeh test prove karta hai ki tumhara DB lock kaam kar raha hai.

Step	Action	Expected Result
Setup	Merchant create karo 10,000 paise (100 INR) balance ke saath	—
Action	Do simultaneous threads mein 6,000 paise ke 2 alag requests bhejo	—
Check 1	Payouts count karo DB mein	Exactly 1 payout created
Check 2	HTTP status codes check karo	Exactly 1 return 201, 1 return 409/422
Check 3	Balance verify karo	4,000 paise bacha hona chahiye

> *Threading use karo — dono requests truly simultaneous honi chahiye, sequential nahi.*

Test 2 — Idempotency Test

Step	Action	Expected Result
Setup	Merchant with balance banao	—
Call 1	POST /api/v1/payouts/ with Idempotency-Key: "test-key-123"	201 Created, payout object
Call 2	Same POST with same "test-key-123"	200 OK, identical response body
Check	DB mein payouts count karo	Exactly 1 payout — no duplicate
Bonus	25 hours baad same key use karo	409/422 — key expired

Bonus Test Ideas (Optional but Impressive)

- → State machine test: completed payout ko pending mein le jaane ki koshish karo → ValueError expect karo
- → Failed payout fund return test: failure ke baad balance badhna chahiye atomically
- → Stuck payout retry test: 30s se zyada processing mein → retry trigger hona chahiye
- → Balance invariant test: koi bhi payout ke baad SUM(credits) - SUM(debits) = balance verify karo

05 Deliverables — Kya Submit Karna Hai

GitHub Repository

- → Clean commit history — ek commit mein sab mat daal dena
- → README.md with setup instructions (local + docker)
- → Seed script: python manage.py seed
- → At least 2 meaningful tests
- → EXPLAINER.md (sabse important file — isko seriously lo)

Live Deployment

Service	Use For	Free Tier?
Railway	Django + PostgreSQL + Redis + Celery	✅ Yes
Vercel	React Frontend	✅ Yes
Render	Alternative to Railway	✅ Yes
Fly.io	Alternative	✅ Yes

> *Seed data production deployment mein bhi hona chahiye — CTO live test karega.*

EXPLAINER.md — 5 Questions (Filter Point)

Yeh 5 questions mein zyaadatar candidates filter hote hain. Short, specific, code paste karo:

Q#	Question	Kya Likhna Hai
Q1	The Ledger	Balance calculation query paste karo. Why credits/debits model?
Q2	The Lock	select_for_update code paste karo. DB primitive explain karo.
Q3	The Idempotency	Key kaise store/check hoti hai? In-flight scenario kya hota hai?
Q4	The State Machine	failed→completed block kahan hai? Code show karo.
Q5	AI Audit	AI ne jo galat code diya — paste karo. Tune kya pakda aur kya fix kiya?

> *Q5 mein honest raho. Agar sab kuch perfect likha toh CTO ko pata chal jaata hai — aur yeh filter point ban jaata hai.*

06 Dhyan Rakhne Wali Baatein

🚫 Kya Nahi Karna

Galti	Sahi Approach
FloatField ya DecimalField use karna for money	BigIntegerField in paise (integer only)
Balance Python mein calculate karna	DB-level SUM() aggregation
select_for_update() bina nowait	select_for_update(nowait=True) use karo
Sync code for background jobs	Celery + Redis mandatory
Balance ko DB column mein store karna	Balance = derived value, always compute
Status directly assign karna (.status = ...)	transition_to() method use karo
Ek bade commit mein sab daal dena	Clean incremental commits
UI pe zyada time waste karna	Architecture aur correctness pe focus karo

✅ Kya Graded Hoga

Criteria	Weight	Kaise Impress Karo
Money Integrity	High	BigInt, DB aggregation, invariant test
Concurrency	Very High	select_for_update(nowait=True), threading test
Idempotency	High	Unique constraint, in-flight handling, 24h expiry
State Machine	Medium	ALLOWED_TRANSITIONS dict, transition_to() method
Retry Logic	Medium	Celery Beat, exponential backoff, 3 max attempts
EXPLAINER.md	Very High	Short, specific, real code, honest AI audit
Code Quality	Medium	Clean commits, readable code, no hacks
UI/UX	Low	Functional chahiye, pixel-perfect nahi

⏰ Time Planning (10–15 Hours)

Din	Focus Area	Hours	Deliverable
Day 1	Models + Migrations + Seed Script	2–3h	DB ready, merchants seeded
Day 2	Payout API (Lock + Idempotency)	3h	POST /api/v1/payouts/ working
Day 3	Celery Worker + State Machine + Retry	2–3h	Background processing working
Day 4	React Dashboard + Polling	2h	Frontend showing live status
Day 5	Tests + EXPLAINER + Deploy + Submit	2h	GitHub + Live URL ready

07 Architecture Overview

Folder Structure

```
playto-payout/
├── backend/
│   ├── ledger/        ← Merchant, Transaction models
│   ├── payouts/       ← Payout API, State machine, Worker
│   └── core/          ← Django settings, Celery config
├── frontend/
│   └── src/           ← React + Tailwind dashboard
├── docker-compose.yml
├── README.md
└── EXPLAINER.md
```

Data Flow — Payout Request

Step	What Happens	Key Code
1	React form → POST /api/v1/payouts/	Idempotency-Key header set karo
2	Idempotency check (before lock)	existing = Payout.objects.filter(key=...).first()
3	DB lock acquire karo	select_for_update(nowait=True)
4	Balance check karo (inside lock)	Transaction.objects.aggregate(SUM)
5	Payout create + debit transaction	Both inside atomic() block
6	Celery task queue karo	process_payout.apply_async(args=[payout.id])
7	Worker simulate karta hai bank	70% success, 20% fail, 10% hang
8	State update + fund release if failed	Atomic: transition_to() + credit transaction
9	React polls every 3s	GET /api/v1/merchants/{id}/payouts/

Bonus Features (Agar Time Mile)

- ⭐ docker-compose.yml — ek command mein sab start ho (impressive)
- ⭐ Audit log — har state change ka log rakho
- ⭐ Webhook delivery — payout complete hone pe merchant ko notify karo
- ⭐ Event sourcing — transactions ko events ki tarah model karo

> *Bonus sab mat karo — jo genuinely tumhe interesting lagta ho wohi karo. CTO depth prefer karta hai breadth se zyada.*

08 EXPLAINER.md — Draft Answers

Yeh questions ke sample answers hain. Apne actual code se paste karna — yeh sirf template hai.

Q1 — The Ledger

> *Paste: SUM(credits) - SUM(debits) aggregation query. Explain: stored as BigIntegerField in paise. Never store balance as column — always derive. This ensures invariant can never break.*

Q2 — The Lock

> *Paste: select_for_update(nowait=True) code block. Explain: PostgreSQL row-level exclusive lock. NOWAIT means second concurrent request fails immediately with OperationalError — no deadlock, no stale read.*

Q3 — Idempotency

> *How system knows it has seen a key: unique_together = [(merchant, idempotency_key)] + expiry check before lock. In-flight: DB unique constraint prevents second INSERT — catches IntegrityError, returns 200 with existing payout.*

Q4 — State Machine

> *Paste: ALLOWED_TRANSITIONS dict. Explain: Payout.transition_to() method checks allowed list. FAILED: [] — empty list means zero transitions allowed out. Failed→completed raises ValueError before any DB write.*

Q5 — AI Audit (Template)

> *Example: AI generated select_for_update() without nowait=True. Problem: without NOWAIT, two concurrent requests both block-wait for the lock. First releases — second reads updated balance. But network delay can still cause issues + locks held unnecessarily long. Fix: Added nowait=True + 409 response. This gives deterministic failure which is correct for a payment API.*

09 Submission Checklist

Submit karne se pehle yeh sab verify karo:

#	Item	Status
1	GitHub repo public hai aur clean commit history hai	☐
2	README.md mein local setup instructions hain	☐
3	python manage.py seed kaam karta hai	☐
4	Concurrency test pass ho raha hai	☐
5	Idempotency test pass ho raha hai	☐
6	Live deployment chal raha hai (Railway/Render)	☐
7	Live URL pe seed data hai	☐
8	EXPLAINER.md 5 questions ke saath complete hai	☐
9	React dashboard live status dikha raha hai	☐
10	Balance invariant verify kiya (manual test)	☐
11	docker-compose.yml (bonus)	☐
12	Submission form fill kiya: forms.gle/71gdyG9Kyvddr7Vu6	☐

Submission Form Mein Kya Bharna Hai

- → GitHub repository link
- → Hosted deployment URL
- → Short note: "What are you most proud of?" — yahan genuinely kuch likho (locking implementation? idempotency edge case? clean architecture?)

All the best! Ship it like a pro. 🚀

Questions → sanhik@playto.so